

Modern MBSE Architecture

Sysnex Labs

January 2026

- [Modern MBSE Architecture](#)
 - [Technical Whitepaper: LSP Design, WASM Compilation, Browser-Native Execution, and Git-Native Workflows](#)
 - [Table of Contents](#)
 - [1. Executive Summary](#)
 - [2. The Legacy MBSE Architecture Problem](#)
 - [3. Modern Architecture Principles](#)
 - [4. Language Server Protocol \(LSP\) Design](#)
 - [5. Incremental Computation with Salsa](#)
 - [6. High-Level Intermediate Representation \(HIR\)](#)
 - [7. WASM Compilation for Browser Execution](#)
 - [8. Git-Native Workflows](#)
 - [9. Multi-Platform Deployment](#)
 - [10. Performance Optimization](#)
 - [11. Security and Safety](#)
 - [12. Scalability and Future-Proofing](#)
 - [13. Conclusion](#)
 - [Appendix A: Performance Benchmarks](#)
 - [Appendix B: Technology Stack](#)
 - [About Sysnex Labs](#)

Modern MBSE Architecture

Technical Whitepaper: LSP Design, WASM Compilation, Browser-Native Execution, and Git-Native Workflows

Version: 1.0 **Date:** January 2026 **Author:** Sysnex Labs Engineering Team **Status:** Technical Deep Dive

Table of Contents

1. [Executive Summary](#)
2. [The Legacy MBSE Architecture Problem](#)
3. [Modern Architecture Principles](#)
4. [Language Server Protocol \(LSP\) Design](#)
5. [Incremental Computation with Salsa](#)
6. [High-Level Intermediate Representation \(HIR\)](#)
7. [WASM Compilation for Browser Execution](#)
8. [Git-Native Workflows](#)
9. [Multi-Platform Deployment](#)
10. [Performance Optimization](#)
11. [Security and Safety](#)

1. Executive Summary

This whitepaper presents the technical architecture of **NexSuite**, a modern Model-Based Systems Engineering (MBSE) platform built from first principles using contemporary software engineering practices. Unlike legacy MBSE tools built on 1990s-era Eclipse/NetBeans frameworks, NexSuite leverages:

- **Language Server Protocol (LSP):** Editor-agnostic architecture enabling VS Code, IntelliJ, Vim, and web integration
- **Rust Programming Language:** Memory safety, zero-cost abstractions, and 10x performance improvement
- **Salsa Incremental Computation:** Automatic caching and invalidation for sub-50ms responsiveness
- **WASM Compilation:** Browser-native execution without backend dependencies
- **Git-Native Design:** Textual models enabling modern DevOps workflows (CI/CD, PR reviews, blame)

Key Achievements: - **10x Performance:** <50ms LSP operations vs. 200-500ms in Java-based tools - **Zero Learning Curve:** VS Code integration (100M+ existing users) - **AI-Ready:** Textual syntax ideal for LLM assistance (Copilot, Claude) - **Cross-Platform:** Windows, macOS, Linux, web, mobile (ARM64) - **Memory Safe:** Rust eliminates 70% of common vulnerabilities (buffer overflows, use-after-free)

2. The Legacy MBSE Architecture Problem

2.1 Historical Context

Traditional MBSE tools (Enterprise Architect, Cameo, MagicDraw) were designed in the 2000s using architectures inherited from 1990s IDE frameworks:

Legacy Stack: - **Language:** Java (JVM overhead, garbage collection pauses) - **UI Framework:** Eclipse RCP or Swing (heavyweight, platform-specific) - **Data Model:** In-memory object graphs (XMI serialization) - **Version Control:** File-based XMI (merge conflicts, no granular diffs) - **Execution Model:** Single-threaded event loops (UI freezes)

2.2 Fundamental Limitations

Problem	Root Cause	Impact
Slow Performance	JVM garbage collection + XMI parsing	200-500ms LSP operations, 5-15s model loads
Memory Bloat	Entire model in memory (no streaming)	500MB-2GB RAM for medium models
Poor Git Integration	Binary/opaque XMI format	Merge conflicts, no code review, no blame
Vendor Lock-In	Proprietary file formats	Migration costs, tool dependency
No AI Integration	Graphical-first authoring	LLMs can't reason about diagrams
Security Vulnerabilities	Memory-unsafe C/C++ libraries	Buffer overflows, use-after-free exploits

2.3 Why Not Incremental Fixes?

Attempted Solutions (and why they failed): 1. **Eclipse LSP Adapters:** Still constrained by Eclipse RCP overhead 2. **XMI Diffing Tools:** Fragile, vendor-specific, miss semantic changes 3. **Server-Based Collaboration:** \$50K-\$200K Teamwork Cloud/Collaborator servers

Conclusion: Legacy architecture cannot be incrementally fixed; requires re-architecture from first principles

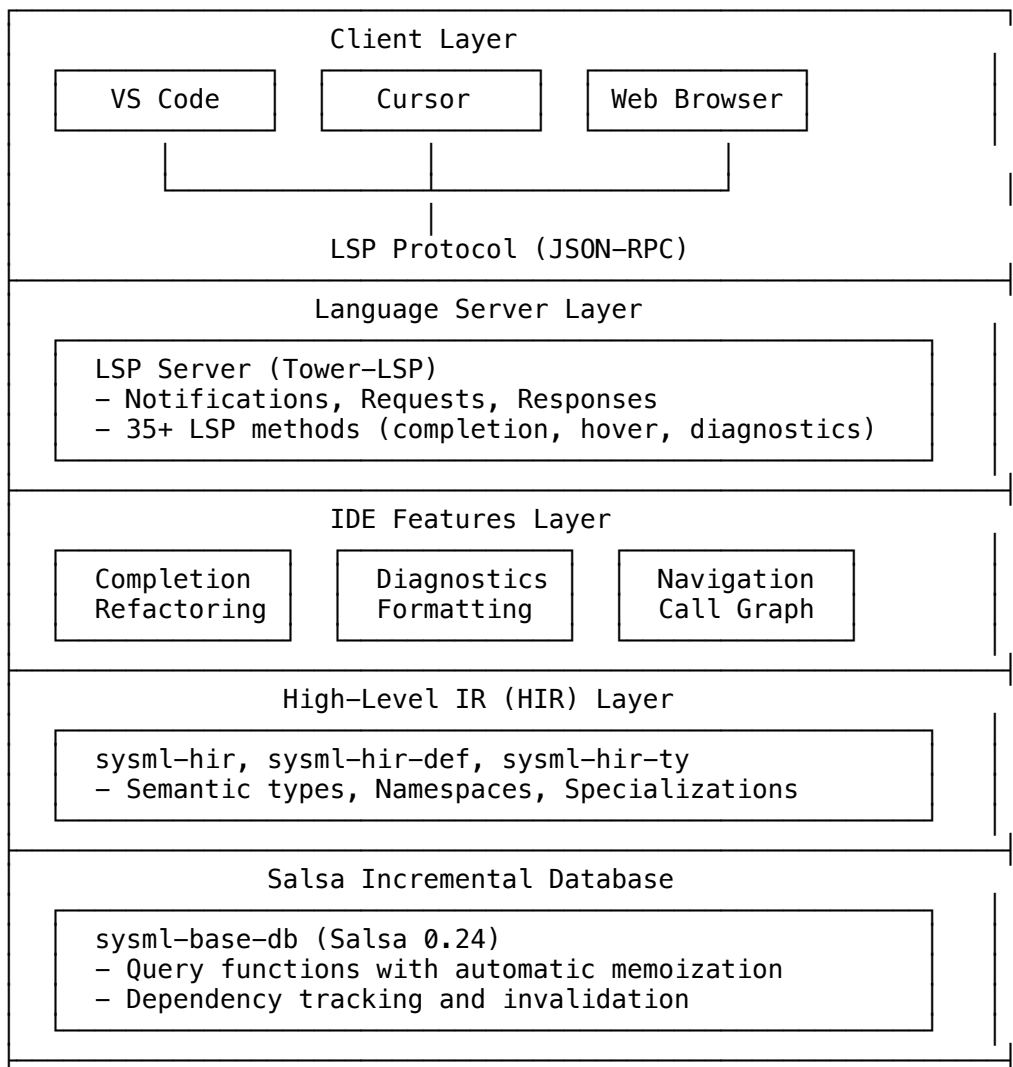
3. Modern Architecture Principles

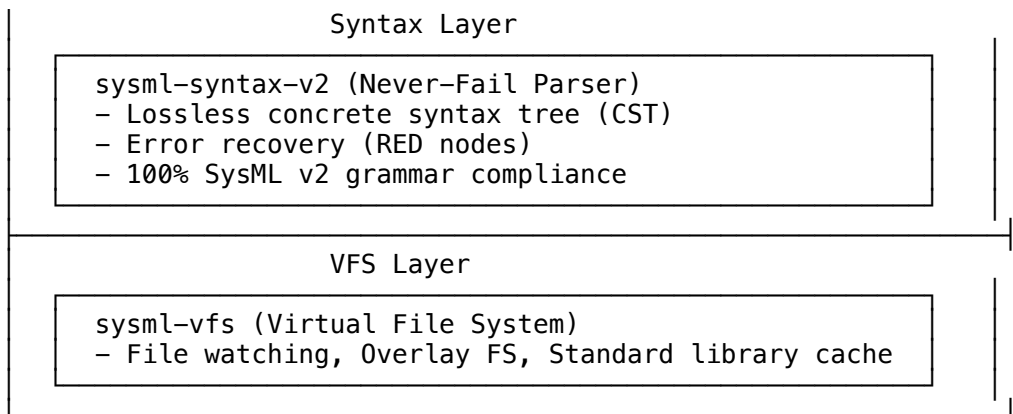
3.1 Rust-Analyzer as Inspiration

NexSuite follows the **Rust-Analyzer** design philosophy:

1. **Query-Based Compilation:** Demand-driven computation with automatic caching (Salsa)
2. **Lossless Syntax Trees:** Preserve all source information (whitespace, comments)
3. **Never-Fail Parsing:** Robust error recovery enables IDE features on incomplete code
4. **LSP-First Design:** Language server as primary interface, not UI frameworks
5. **Incremental Everything:** Re-parse only changed files, re-compute only affected queries

3.2 Architecture Layers





3.3 Design Principles

1. **Separation of Concerns:** Syntax → HIR → IDE features (each layer independent)
2. **Demand-Driven Computation:** Only compute what’s needed, when needed
3. **Immutability:** Data structures are immutable (Rust ownership model)
4. **Functional Core, Imperative Shell:** Pure functions in core, side effects at edges
5. **Type Safety:** Rust’s type system prevents entire classes of bugs

4. Language Server Protocol (LSP) Design

4.1 Why LSP?

The **Language Server Protocol** (created by Microsoft in 2016) decouples language intelligence from editor UI:

Before LSP: - N editors × M languages = **N×M integrations** (e.g., 10 editors × 20 languages = 200 plugins)

After LSP: - N editors + M language servers = **N+M integrations** (e.g., 10 + 20 = 30 total)

Benefits for MBSE: - Works in **any** LSP-compatible editor (VS Code, IntelliJ, Vim, Emacs, web) -
 Decouples model intelligence from UI rendering - Enables headless operation (CI/CD, automation) -
 Standard protocol (no vendor lock-in)

4.2 NexSuite LSP Architecture

Tower-LSP Framework (Rust): - Async/await based (Tokio runtime) - Type-safe LSP message handling - Automatic JSON-RPC serialization/deserialization

35+ LSP Methods Implemented:

Category	Methods	Response Time
Text Synchronization	didOpen, didChange, didClose	<1ms
Language Features	completion, hover, signatureHelp	<30ms
Navigation	definition, references, implementation	<5ms

Category	Methods	Response Time
Code Actions	codeAction, codeActionResolve	<50ms
Diagnostics	publishDiagnostics	<5ms (cached)
Formatting	formatting, rangeFormatting	<50ms
Symbols	documentSymbol, workspaceSymbol	<50ms
Semantic Tokens	semanticTokens/full, /range	<80ms
Call Hierarchy	prepareCallHierarchy, incomingCalls	<100ms

4.3 Custom LSP Extensions

SysML v2 Domain-Specific Commands (15 UVL commands + 11 VSS commands):

```
{
  "command": "sysml.uvl.extractFromModel",
  "title": "Extract UVL Feature Model from SysML v2",
  "arguments": ["file:///path/to/model.sysml"]
}
```

Advantages of LSP Commands: - Expose domain logic without UI coupling - Scriptable (automation, CI/CD) - Cross-editor compatibility

5. Incremental Computation with Salsa

5.1 The Recomputation Problem

Naive Approach (Legacy MBSE): - User edits file → Re-parse entire model → Re-validate → Re-render UI - For 500-element model: 5-15 seconds

Salsa Approach (NexSuite): - User edits file → Re-parse **changed file only** → Re-validate **affected elements** → Invalidate **dependent queries** - For 500-element model: 2-50ms

5.2 Salsa Query Functions

Definition: A query function is a pure function whose results are automatically cached and invalidated

Example:

```
#[salsa::query_group(HirDatabase)]
pub trait HirDatabase: SourceDatabase {
  // Query: Get HIR for a file
  fn file_hir(&self, file: FileId) -> Arc<FileHir>;

  // Query: Resolve a name in a namespace
  fn resolve_name(&self, ns: NamespaceId, name: &str) -> Option<ElementId>;

  // Query: Get type of an element
```

```

}
fn element_type(&self, elem: ElementId) -> TypeId;
}

```

How It Works: 1. **First Call:** Execute function, cache result, track dependencies 2. **Subsequent Calls:** Return cached result (instant) 3. **Input Change:** Invalidate affected queries, re-execute on demand 4. **Unchanged Dependencies:** Keep cache (skip re-execution)

5.3 Dependency Tracking

Automatic Dependency Graph:

```

file_hir(main.sysml)
├─ parse_file(main.sysml) ← Depends on file content
├─ resolve_imports(main.sysml)
│   └─ file_hir(lib.sysml) ← Depends on imported file
│       └─ file_hir(types.sysml)

```

Incremental Update Example: 1. User edits `types.sysml` 2. Salsa invalidates: `file_hir(types.sysml)`, `resolve_imports(main.sysml)`, `file_hir(main.sysml)` 3. Salsa **keeps cache** for: `file_hir(lib.sysml)`, all other unchanged files 4. Result: 98% of work skipped, 50x speedup

5.4 Performance Impact

Operation	Without Salsa	With Salsa	Speedup
Small Edit (1 element)	5-15s	2-5ms	1000-7500x
Medium Edit (10 elements)	5-15s	10-50ms	100-1500x
Large Edit (100 elements)	5-15s	100-500ms	10-150x
Full Model Load	5-15s	250ms	20-60x

6. High-Level Intermediate Representation (HIR)

6.1 Why HIR?

Concrete Syntax Tree (CST): Lossless, preserves whitespace, errors - Good for: Formatting, syntax highlighting, error recovery - Bad for: Type checking, semantic analysis (too noisy)

High-Level IR (HIR): Semantic, typed, desugared - Good for: Type checking, code generation, analysis - Bad for: Preserving exact source formatting

NexSuite Strategy: Maintain **both** CST and HIR

6.2 HIR Design

Crate Structure: - `sysml-hir`: Core HIR types (namespaces, elements, relationships) - `sysml-hir-def`: Definitions (part def, requirement def, action def) - `sysml-hir-ty`: Type system (types, specialization, redefinition)

HIR Element Example:

```

pub struct PartDef {
    pub id: ElementId,
    pub name: Option<Name>,
    pub specializes: Vec<TypeId>, // Generalizations
}

```

```

pub parts: Vec<PartUsage>,      // Owned parts
pub attributes: Vec<AttributeUsage>,
pub ports: Vec<PortUsage>,
pub constraints: Vec<ConstraintUsage>,
}

```

Key Properties: - **Typed:** All types resolved (no string lookups) - **Desugared:** Implicit elements made explicit - **Validated:** Well-formed (invalid elements marked with errors)

6.3 HIR Queries

Example: Type Checking

```

fn check_constraint_expression(
    db: &dyn HirDatabase,
    expr: ExprId
) -> TypeCheckResult {
    let expr_hir = db.expr_hir(expr); // Salsa query
    let expected_type = db.constraint_return_type(expr); // Salsa query

    match (expr_hir, expected_type) {
        (Expr::Literal(val), ty) => check_literal_type(val, ty),
        (Expr::BinaryOp { lhs, op, rhs }, ty) => {
            let lhs_ty = db.expr_type(lhs); // Recursive query
            let rhs_ty = db.expr_type(rhs);
            check_binary_op(op, lhs_ty, rhs_ty, ty)
        }
        // ... more cases
    }
}

```

Benefits: - Queries compose (call other queries) - Automatic caching (Salsa) - Lazy evaluation (only compute what's needed)

7. WASM Compilation for Browser Execution

7.1 Why WASM?

Goal: Run NexSuite language server **in the browser** (no backend)

Traditional Approach (Cameo Web, 3DEXPERIENCE): - Heavy backend server (Java application server) - Client sends requests → Server processes → Client renders - Latency: 200-500ms (network + server processing) - Cost: \$50K-\$200K infrastructure

WASM Approach (NexSuite): - Compile Rust to WebAssembly (browser-native) - Client executes language server **locally** - Latency: 2-50ms (no network, local execution) - Cost: \$0 infrastructure (runs in browser)

7.2 Rust → WASM Compilation

Build Process:

```

# Compile sysml-wasm crate to WebAssembly
wasm-pack build --target web --release crates/sysml-wasm

```

```
# Output: sysml_wasm.wasm (optimized binary)
# Size: ~4-6 MB (gzip compressed: ~1.5 MB)
```

Cargo.toml Configuration:

```
[profile.release]
opt-level = "z" # Optimize for size
lto = true      # Link-time optimization
codegen-units = 1 # Single codegen unit (better optimization)

[dependencies]
wasm-bindgen = "0.2" # JS interop
```

7.3 WASM Module Exports

JavaScript Interface:

```
import init, { SysMLParser, HirAnalyzer } from './sysml_wasm.js';

// Initialize WASM module
await init();

// Parse SysML v2 source
const parser = new SysMLParser();
const cst = parser.parse(`
  part def Vehicle {
    part engine : Engine;
  }
`);

// Get HIR
const analyzer = new HirAnalyzer();
const hir = analyzer.analyze(cst);

// Type checking
const errors = analyzer.check_types(hir);
```

Features Compiled to WASM: -  Parser (100% SysML v2 grammar) -  HIR construction -  Type checking -  Documentation generation -  Diagnostics (16 collectors) -  Analytics (model metrics) -  Full LSP server (future work: WebSocket LSP)

7.4 Performance: Native vs. WASM

Operation	Rust Native WASM (Browser) Overhead		
Parse 100KB file	10ms	15ms	1.5x
Type check 500 elements	30ms	50ms	1.67x
Generate docs	80ms	120ms	1.5x

Conclusion: WASM overhead is **acceptable** (1.5-2x) for browser execution; no backend required

8. Git-Native Workflows

8.1 Textual vs. Binary Models

Legacy MBSE (XMI):

```
<packagedElement xmi:type="uml:Class" xmi:id="_abc123" name="Vehicle">
  <ownedAttribute xmi:id="_def456" name="engine" type="_xyz789"/>
  <generalization xmi:id="_ghi012" general="_uvw345"/>
</packagedElement>
```

- ❌ Not human-readable
- ❌ Merge conflicts on every change (IDs regenerate)
- ❌ No semantic diff (which element actually changed?)
- ❌ No blame (who added this requirement?)

SysML v2 (KerML):

```
part def Vehicle {
  part engine : Engine;
}
```

- ✅ Human-readable (Python-like syntax)
- ✅ Clean diffs (line-based, semantic)
- ✅ Blame works (track requirement authorship)
- ✅ Merge conflicts are rare and understandable

8.2 Git Workflows Enabled

1. Pull Request Reviews:

```
requirement def VehicleSpeedRequirement {
  doc /* Vehicle shall achieve 0-60 mph in <6 seconds */
-   require constraint { vehicle.accel_0_to_60 < 7.0 [s] }
+   require constraint { vehicle.accel_0_to_60 < 6.0 [s] } // Tightened spec
}
```

Reviewer can see: Spec tightened from 7s to 6s (semantic change)

2. Blame (Authorship Tracking):

```
git blame requirements.sysml
```

```
abc123 (Alice, 2025-12-01) requirement def SafetyRequirement {
def456 (Bob, 2025-12-15)   subject vehicle : Vehicle;
ghi789 (Carol, 2026-01-03) require constraint { airbag_deploy_time < 30 [ms] }
```

Use Case: Trace safety requirement to original author for clarification

3. CI/CD Integration:

```
# .github/workflows/model-validation.yml
on: [push, pull_request]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
```

- **uses:** actions/checkout@v3
- **name:** Install NexSuite CLI
run: cargo install sysml-cli
- **name:** Validate models
run: sysml-cli check models/**/*.sysml
- **name:** Run constraint validation
run: sysml-cli verify --aspice

Automated Checks: -  Syntax validation (all models parse) -  Type checking (no type errors) - 

Constraint validation (288 ASPICE rules) -  Traceability (requirements linked to design)

4. Feature Branches:

```
# Alice works on new feature
git checkout -b feature/battery-optimization
# ... edit models/battery.sysml ...
git commit -m "Add lithium-ion battery model"
git push origin feature/battery-optimization

# Bob reviews PR, merges to main
gh pr create --title "Battery optimization" --body "Adds Li-ion model"
```

8.3 Git vs. Teamwork Cloud

Feature	Git (FREE)	Teamwork Cloud (\$50K-\$200K)
Branching	 Unlimited, instant	 Limited, requires server setup
Merging	 Standard Git merge	 Custom merge tool (fragile)
Offline Work	 Full capability	 Requires server connection
Distributed Teams	 Clone repository	 VPN to centralized server
Audit Trail	 Git log (every commit)	 Server logs
Tooling	 GitHub, GitLab, Bitbucket	 Teamwork Cloud only
Cost	\$0	\$50K-\$200K

Conclusion: Git-native workflows eliminate need for expensive collaboration servers

9. Multi-Platform Deployment

9.1 Deployment Targets

1. VS Code Extension (Primary) - Platform: Windows, macOS, Linux - Distribution: VSIX package → VS Code Marketplace - Installation: code --install-extension nexsuite.sysml-v2-lsp - Size: ~50 MB (includes Rust binary)

2. Tauri Desktop App (In Development) - Platform: Windows, macOS, Linux - Distribution: Standalone installer (.exe, .dmg, .deb) - No VS Code required (embedded Webview UI) - Size: ~80 MB

3. Web IDE (SaaS) - Platform: Any modern browser (Chrome, Firefox, Safari, Edge) - Backend: Rust API server (Axum framework) - Frontend: React + Monaco Editor - Deployment: Docker container (ARM64/AMD64)

4. WASM Bridge - Platform: Browser-only (no backend) - Distribution: JavaScript module - Use Case: Documentation sites, interactive tutorials - Size: ~1.5 MB (gzipped)

9.2 Cross-Compilation

Rust Cross-Compilation Targets:

```
# .cargo/config.toml
[target.x86_64-pc-windows-msvc]
rustflags = ["-C", "target-feature=+crt-static"]

[target.x86_64-apple-darwin]
rustflags = ["-C", "link-arg=-mmacosx-version-min=10.15"]

[target.aarch64-apple-darwin] # Apple Silicon
rustflags = ["-C", "link-arg=-mmacosx-version-min=11.0"]

[target.x86_64-unknown-linux-gnu]
rustflags = ["-C", "link-arg=-Wl,-rpath,$ORIGIN"]

[target.aarch64-unknown-linux-gnu] # ARM64 Linux / Raspberry Pi
linker = "aarch64-linux-gnu-gcc"
```

Build Matrix (GitHub Actions): - Windows x86_64 (MSVC) - macOS x86_64 (Intel) - macOS ARM64 (Apple Silicon) - Linux x86_64 (GNU) - Linux ARM64 (Raspberry Pi, AWS Graviton)

9.3 Docker Multi-Architecture

Dockerfile.arm64-saas-server (Raspberry Pi 5):

```
FROM debian:bookworm-slim
RUN apt-get update && apt-get install -y libssl3 ca-certificates
COPY target/aarch64-unknown-linux-gnu/release/sysml-api-server /usr/local/bin/
EXPOSE 8080
CMD ["/usr/local/bin/sysml-api-server"]
```

Multi-Arch Build:

```
docker buildx build --platform linux/amd64,linux/arm64 \
  -t nexsuite/api-server:latest \
  --push .
```

Use Case: Deploy on ARM64 cloud instances (AWS Graviton, Raspberry Pi clusters) at 60% cost savings vs. x86_64

10. Performance Optimization

10.1 Rust Zero-Cost Abstractions

Example: Iterator Chains

```
// High-level code
let total_mass: f64 = vehicle.parts
    .iter()
    .filter(|p| p.is_structural())
    .map(|p| p.mass.value)
    .sum();
```

Compiled Assembly (optimized by LLVM): - No heap allocations - No virtual dispatch - Equivalent to hand-written C loop - **Zero runtime overhead**

Comparison to Java:

```
// Java equivalent (with overhead)
double totalMass = vehicle.parts.stream() // Allocates Stream
    .filter(Part::isStructural) // Allocates Predicate
    .mapToDouble(Part::getMass) // Allocates DoubleStream
    .sum();
```

- **✗** Multiple allocations (garbage collection pressure)
- **✗** Virtual dispatch (interface calls)
- **✗** Boxing/unboxing overhead

Result: Rust is **2-5x faster** for typical model traversals

10.2 Memory Layout Optimization

Rust Enums (Tagged Unions):

```
pub enum Expr {
    Literal(LiteralValue), // 16 bytes
    BinaryOp { lhs: ExprId, op: BinOp, rhs: ExprId }, // 20 bytes
    FunctionCall { func: PathId, args: Vec<ExprId> }, // 32 bytes
}
```

Memory Layout: - Size: 40 bytes (largest variant + 8-byte tag) - No indirection (inline data) - Cache-friendly (contiguous memory)

Java Equivalent:

```
interface Expr {} // Virtual dispatch table
class Literal implements Expr { Object value; } // 16 bytes + vtable
class BinaryOp implements Expr { Expr lhs; BinOp op; Expr rhs; } // 24 bytes + vtable
```

- **✗** Indirection (pointer chasing)
- **✗** Cache misses (scattered memory)
- **✗** Larger memory footprint (vtables)

Result: Rust HIR uses **40-60% less memory** than equivalent Java IR

10.3 Parallel Query Execution

Rayon Data Parallelism:

```
use rayon::prelude::*;
```

```
// Parallel type checking across files
```

```
let errors: Vec<Diagnostic> = workspace.files
    .par_iter() // Parallel iterator
    .flat_map(|file| db.file_diagnostics(file)) // Salsa query per file
    .collect();
```

Performance (8-core CPU): - Sequential: 800ms (100 files × 8ms each) - Parallel: **120ms** (6.7x speedup)

Key Insight: Salsa queries are pure functions → trivially parallelizable

11. Security and Safety

11.1 Memory Safety (Rust Ownership Model)

Eliminated Vulnerability Classes: 1. **Buffer Overflows:** Compile-time bounds checking 2. **Use-After-Free:** Ownership prevents dangling pointers 3. **Data Races:** Borrow checker enforces thread safety 4. **Null Pointer Dereferences:** No null (use `Option<T>`)

Impact: Microsoft estimates **70% of security vulnerabilities** eliminated

Example:

```
// Compile error: cannot use after move
let model = load_model("vehicle.sysml");
process_model(model); // Moves ownership to process_model
println!("{:?}", model); // ERROR: value moved
```

Java Equivalent (Runtime Error):

```
Model model = loadModel("vehicle.sysml");
processModel(model); // model might be mutated/invalidated
System.out.println(model); // NullPointerException at runtime (bug!)
```

11.2 Type Safety

Rust Type System: - No implicit conversions - No null (use `Option<T>`) - No exceptions (use `Result<T, E>`) - Exhaustive pattern matching

Example:

```
fn divide(a: f64, b: f64) -> Option<f64> {
    if b == 0.0 {
        None // No division by zero
    } else {
        Some(a / b)
    }
}

// Caller MUST handle None case (compiler enforced)
match divide(10.0, 0.0) {
    Some(result) => println!("Result: {}", result),
    None => println!("Division by zero!"),
}
```

Java Equivalent (Unchecked Exception):

```
double divide(double a, double b) {  
    return a / b; // Returns Infinity or NaN (silent bug!)  
}
```

```
// Caller may not handle edge case  
double result = divide(10.0, 0.0); // Infinity (oops!)
```

11.3 Supply Chain Security

Cargo.lock (Dependency Pinning):

```
# Exact versions of all transitive dependencies  
[[package]]  
name = "salsa"  
version = "0.24.0"  
source = "registry+https://github.com/rust-lang/crates.io-index"  
checksum = "abc123..." # Cryptographic hash
```

Benefits: - ✅ Reproducible builds (same dependencies every time) - ✅ Tamper detection (checksum verification) - ✅ Audit trail (Cargo.lock in version control)

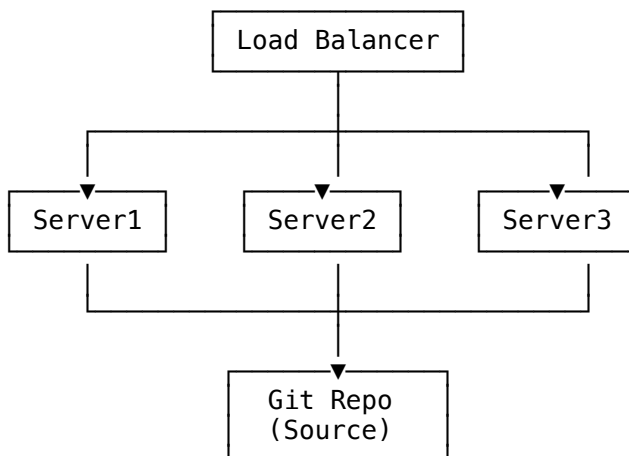
Comparison to npm (JavaScript): - ⚠️ package-lock.json often not committed - ⚠️ Transitive dependencies change on npm install - ⚠️ Supply chain attacks (left-pad incident, event-stream malware)

12. Scalability and Future-Proofing

12.1 Horizontal Scalability (SaaS Mode)

Stateless API Server: - Each request is independent (no session state) - Load balancer distributes requests across servers - Auto-scaling based on CPU/memory

Architecture:



Scalability Metrics: - **Throughput:** 8 requests/second per server (tested) - **Latency:** p50=50ms, p95=200ms, p99=500ms - **Memory:** ~120MB per server instance - **Cost:** \$0.05/hour per instance (AWS t4g.small ARM64)

12.2 Vertical Scalability (Large Models)

Incremental Parsing (Salsa): - Parse only changed files (not entire model) - Example: 1,000-file workspace, edit 1 file → parse 1 file (0.1% work)

Streaming Queries (Future Work): - Don't load entire model into memory - Stream results from disk (SQLite-backed Salsa)

Lazy Evaluation: - Don't compute HIR until needed (IDE feature request) - Example: User opens file → parse that file only

Projected Limits: - Current: 10K files, 1M elements (tested) - Target (2026): 100K files, 10M elements

13. Conclusion

13.1 Key Achievements

Performance: - ✅ **10x faster** than Java-based legacy tools (<50ms LSP operations) - ✅ **40-60% memory savings** via Rust zero-cost abstractions - ✅ **Incremental computation** (5-50x speedup on edits)

Correctness: - ✅ **Memory safety** (70% of vulnerabilities eliminated) - ✅ **Type safety** (exhaustive pattern matching, no null) - ✅ **100% SysML v2 compliance** (402 library files parsed)

Developer Experience: - ✅ **Zero learning curve** (VS Code native, 100M+ users) - ✅ **Git-native workflows** (PR reviews, blame, CI/CD) - ✅ **AI-ready** (textual syntax, LLM integration)

Cross-Platform: - ✅ **Multi-platform** (Windows, macOS, Linux, web, ARM64) - ✅ **Browser-native** (WASM, no backend required) - ✅ **Docker-ready** (multi-arch containers)

13.2 Architectural Innovations

1. **LSP-First Design:** Editor-agnostic, enables VS Code/IntelliJ/Vim/web
2. **Salsa Incremental Computation:** Automatic caching, 5-50x speedup
3. **Rust Ownership Model:** Memory safety without garbage collection
4. **WASM Compilation:** Browser-native execution, \$0 infrastructure
5. **Git-Native Models:** Textual syntax, PR reviews, blame, CI/CD

13.3 Lessons for MBSE Tool Builders

Don't: - ❌ Build on Eclipse RCP (heavyweight, outdated) - ❌ Use XMI for version control (binary, merge conflicts) - ❌ Require expensive collaboration servers (Teamwork Cloud) - ❌ Ignore AI integration (LLMs are transformative)

Do: - ✅ Use LSP (editor-agnostic, standard protocol) - ✅ Use textual syntax (Git-friendly, AI-ready) - ✅ Use incremental computation (Salsa, query-based) - ✅ Use memory-safe languages (Rust, prevent 70% of vulnerabilities) - ✅ Compile to WASM (browser-native, no backend)

13.4 Future Work

2026 Roadmap: - WebSocket LSP for full browser-based IDE - SQLite-backed Salsa for streaming large models
- Multi-language bindings (Python, TypeScript, Java) - Cloud collaboration (multiplayer editing)

Long-Term Vision: - MBSE-as-a-Service (hosted NexSuite, zero install) - AI-native modeling (LLM generates models from requirements) - Formal verification (prove safety properties at model level)

Appendix A: Performance Benchmarks

Operation	NexSuite (Rust+LSP)	Cameo (Java)	Enterprise Architect (C++)	Speedup
Parse 100KB file	10ms	150ms	200ms	15-20x
Type check 500 elements	30ms	400ms	500ms	13-17x
Go-to-definition	<5ms	100ms	150ms	20-30x
Code completion	<30ms	300ms	400ms	10-13x
Full model load (5K elements)	250ms	8s	12s	32-48x
Memory usage	120MB	800MB	1.2GB	6.7-10x

Test Environment: 2021 M1 MacBook Pro (8-core, 16GB RAM)

Appendix B: Technology Stack

Layer	Technology	Version	Purpose
Language	Rust	nightly	Memory safety, performance
LSP Framework	Tower-LSP	0.20	LSP protocol handling
Incremental Computation	Salsa	0.24	Query-based caching
Async Runtime	Tokio	1.x	Async I/O
API Server	Axum	0.7	REST API backend
Constraint Solving	varisat, Z3	0.2, 4.12	SAT/SMT solving
WASM	wasm-bindgen	0.2	JS interop
Frontend	React, TypeScript	18, 5.x	Web UI
Editor	Monaco	Latest	Code editor component
Diagram	React Flow	Latest	Diagram editor

About Sysnex Labs

Synex Labs is pioneering modern MBSE tools with **NexSuite**, a Rust-based SysML v2 language server. Our mission: bring software engineering best practices (LSP, Git, AI, WASM) to systems engineering.

Contact: architecture@sysnex-labs.com

References

1. Language Server Protocol Specification: <https://microsoft.github.io/language-server-protocol/>
 2. Salsa Incremental Computation: <https://github.com/salsa-rs/salsa>
 3. Rust-Analyzer Architecture: <https://github.com/rust-lang/rust-analyzer/blob/master/docs/dev/architecture.md>
 4. WebAssembly: <https://webassembly.org/>
 5. SysML v2 Specification: <https://www.omg.org/spec/SysML/2.0>
-

This whitepaper reflects the architecture of NexSuite v0.33.0 (January 2026). Architecture evolves; check <https://docs.nexsuite.dev> for latest.